KHANA KHAZANA

Smart POS System

Comprehensive Project Documentation

A Restaurant Point-of-Sale System

Built with ASP.NET | 3-Tier Architecture | Real-time Order Management

Developed By	Mr. Sarthak Patel
Project Title	Khana Khazana: Smart POS System
Technology	ASP.NET, C#, SQL Server
Architecture	3-Tier (Presentation, BLL, DAL)
Institution	Shree Ramkrishna Institute of Computer Education and Applied Sciences
University	Sarvajanik University, Surat
Internal Guide	Prof. Shruti Revdiwala
External Guide	Mr. Nilesh Desai — Desai Software
Project Duration	6 Months (Nov 2024 – May 2025)

Executive Summary

Khana Khazana is a full-featured Smart Point of Sale (POS) system developed specifically for the restaurant industry. The system eliminates the manual overhead of paper-based order tracking, reduces billing errors, and provides real-time visibility into table status, order flow, and payment processing.

The platform serves four distinct user roles — Admin, Waiter, Chef, and Cashier — each with a purpose-built interface tailored to their workflow. Admins configure the entire system including menu items, table layouts, and user accounts. Waiters take table-side orders from a clean, image-rich menu. Chefs receive Kitchen Order Tickets (KOT) in real time. Cashiers handle billing, discounts, and payment acceptance across Cash, Bill printing, and E-Wallet/UPI gateways.

Built on a robust 3-tier architecture using ASP.NET (.NET Framework) and C#, the system cleanly separates the Presentation Layer, Business Logic Layer, and Data Access Layer. This ensures maintainability, security (parameterised SQL queries, role-based access), and scalability for future features such as inventory management, customer ordering portals, and automated reporting.

1. Project Overview

1.1 Company Profile — Desai Software

Company Name	Desai Software
Director	Mr. Nilesh Desai
Business Type	Software Development, Web Development, Mobile App Development, Digital Marketing
Address	206, Elita Square, Nr. VIP Circle, Mota Varachha, Utran, Surat – 394105, Gujarat, India
Website	https://desaisoftware.com
Key Products	Desai Barcode Inventory System | Desai Account Management System | Desai Garment Management System

1.2 Project Profile

Project Title	Khana Khazana: Smart POS System
Application Type	Web Application
Technology Stack	ASP.NET (.NET Framework), C#, SQL Server / MySQL
IDE	Visual Studio 2022
Project Duration	6 Months
Internal Guide	Prof. Shruti Revdiwala
External Guide	Mr. Nilesh Desai — Desai Software
Developer	Sarthak P. Patel

1.3 Problem Statement

Traditional restaurants rely heavily on handwritten order slips, paper-based billing, and verbal communication between waiters and kitchen staff. This approach leads to:

- Order errors due to illegible handwriting or miscommunication
- Delays in billing resulting in poor customer experience
- Inability to track real-time table occupancy and revenue
- No centralised view of menu availability or stock levels
- High dependency on manual staff for every transaction

1.4 Proposed Solution

Khana Khazana provides a unified digital ecosystem where all restaurant operations converge. The system provides:

- Role-based dashboards for Admin, Waiter, Chef, and Cashier
- Real-time order management with instant KOT generation
- Digital menu management with image uploads and category filtering
- Automated billing with 5% tax calculation and multiple payment modes
- Table status monitoring across multiple seating zones

2. System Introduction

2.1 System Definition

Khana Khazana is a web-based, role-aware Point of Sale system for restaurants. It provides a seamless ordering lifecycle from table assignment through to payment — without any paper. The system manages menus, orders, kitchen tickets, billing, and user accounts in one integrated platform accessible from any modern browser.

2.2 System Objectives

- Provide a complete POS solution covering all restaurant operations
- Eliminate manual paperwork and reduce human error
- Enable real-time table status monitoring and order tracking
- Automate billing with support for multiple payment methods
- Implement role-based security so each user sees only their workflow
- Deliver a responsive UI that works on desktops, tablets, and mobile devices
- Lay a scalable foundation for future inventory and reporting modules

2.3 User Roles & Responsibilities

Admin

- Configure and manage menu categories and individual menu items
- Define table categories (Garden, Cafeteria, Family, AC, VIP) and table layouts
- Manage user accounts and assign roles
- Access the full dashboard, counter, KOT, and Master Setup sections

Waiter

- Browse the table map and select the correct table for a customer
- Browse the digital menu by category or via the All Dishes view
- Add items to the order, adjust quantities, and submit the order to the kitchen
- Request bill generation or mark items as service-complete

Cashier (Counter)

- View all tables with live order status and billing amounts
- Generate and print customer bills instantly
- Accept payment via Cash, Bill print, or E-Wallet/UPI gateway
- Settle orders and free tables for the next customer

Chef

- View incoming KOT orders in real time
- Mark individual items or full orders as prepared
- Coordinate with front-of-house staff for timely service

3. Requirement Analysis & Modelling

3.1 Functional Requirements

#	Module	Requirement
FR-01	Authentication	Users login with username/password; system validates credentials and redirects based on role
FR-02	Menu Management	Admin can add, edit, and delete menu categories and items with images, prices, display ranks
FR-03	Table Management	Admin defines table categories and individual tables with shape, capacity, and status
FR-04	Order Placement	Waiter/Cashier selects a table, adds items from the menu, and submits the order
FR-05	KOT Generation	On order submission, a Kitchen Order Ticket is instantly sent to the chef view
FR-06	Billing	System auto-calculates subtotal + 5% tax; cashier generates and prints the bill
FR-07	Payment	System supports Cash, Bill-print, and E-Wallet/UPI payment options
FR-08	User Management	Admin can create, edit, and remove user accounts with specific roles
FR-09	VEG/NON-VEG	Menu items carry a green (Veg) or red (Non-Veg) indicator visible on the ordering screen
FR-10	Responsivene ss	Order page is fully responsive and usable on tablets and mobile devices

3.2 Non-Functional Requirements

- Security: Parameterised SQL queries prevent SQL injection; passwords must not be stored in plain text
- Performance: Pages must load within 2 seconds under normal restaurant load
- Usability: Interface must be intuitive for non-technical kitchen and service staff
- Scalability: 3-tier architecture allows independent scaling of UI, logic, and database layers
- Availability: System must be operational during all restaurant service hours with minimal downtime

3.3 Use Case Summary

The system serves four actors. The Admin interacts with Table Management, Menu Management, and User Access Control. Both the Waiter and Cashier interact with Order Management. The Chef handles Order Preparation, and the Cashier manages Billing and Payment Processing.

3.4 Workflow (Flowchart Summary)

The system flow begins at Login, which checks the user role and routes the user to the appropriate module:

- Admin: Choose action → Manage Categories / Menu Items / Tables / Users → Admin End
- Waiter: Take customer order → Place order in system → Order submitted to KOT → Waiter End
- Chef: View KOT orders → Prepare order → Confirm order ready → Chef End
- Cashier: View ready orders → Generate customer bill → Accept payment → Save transaction → Cashier End

3.5 Sequence Diagram Summary

The detailed sequence for a complete order cycle is:

- User places order → Waiter enters details in Frontend → Frontend submits order to Backend
- Backend saves order to Database → Backend sends KOT to Chef
- Chef prepares each item and updates order status to Ready
- Waiter serves food → Customer requests bill
- Cashier generates bill via Frontend → Backend fetches order and calculates total
- Payment processed via gateway → Transaction saved → Order marked complete

4. System Architecture

4.1 Architecture Overview

Khana Khazana follows a clean 3-Tier Architecture, separating the system into three independently maintainable layers:

Presentation Layer — WebApp Project

- Provides the browser-based UI for all four user roles
- Handles HTTP requests, form inputs, and view rendering (ASPX pages)
- Calls the Business Logic Layer exclusively; never touches the database directly

Business Logic Layer — BusinessLogicLibrary

- Contains all business rules: order validation, billing calculations, role checks
- Validates all user inputs before passing them to the Data Access Layer
- Ensures correct business flow (e.g., table must be selected before ordering)

Data Access Layer — DataAccessLibrary

- Manages all database interactions (CRUD operations via stored procedures)
- Uses parameterised queries to prevent SQL injection
- Returns clean data models to the Business Logic Layer

4.2 Technology Stack

Layer	Technology	Purpose
Presentation	ASP.NET Web Forms (.NET Framework)	Browser-based UI — ASPX pages, code-behind in C#
Business Logic	C# Class Library	Business rules, input validation, calculations
Data Access	C# Class Library + ADO.NET	SQL queries, stored procedures, DB connection management
Database	SQL Server / MySQL	Persistent storage for all application data
Frontend	HTML5, CSS3, Bootstrap, jQuery	Responsive layout, interactive controls
IDE	Visual Studio 2022	Development, debugging, and deployment environment

4.3 Data Flow

User (Browser) → Presentation Layer (ASPX + C# Code-Behind) → Business Logic Library → Data Access Library → SQL Server Database

Results flow back in reverse: Database → DAL → BLL → Presentation → User

4.4 Code Guidelines

- Error handling: try-catch blocks in all BLL and DAL methods
- SQL injection prevention: parameterised queries throughout the DAL
- Layering discipline: UI calls BLL only; BLL calls DAL; no direct UI→DAL calls
- Connection management: database connections wrapped in using blocks for proper disposal
- Naming conventions: PascalCase for classes and methods, camelCase for variables
- Password security: passwords must be hashed; no plain-text storage
- Branch strategy: feature/feature-name and bugfix/issue-description

5. Interface Design & Screenshots

5.1 Login Page

The login page provides a secure entry point for all users. The system validates credentials against the database and redirects each user to their role-specific dashboard. An invalid username or password triggers a clear error dialog.

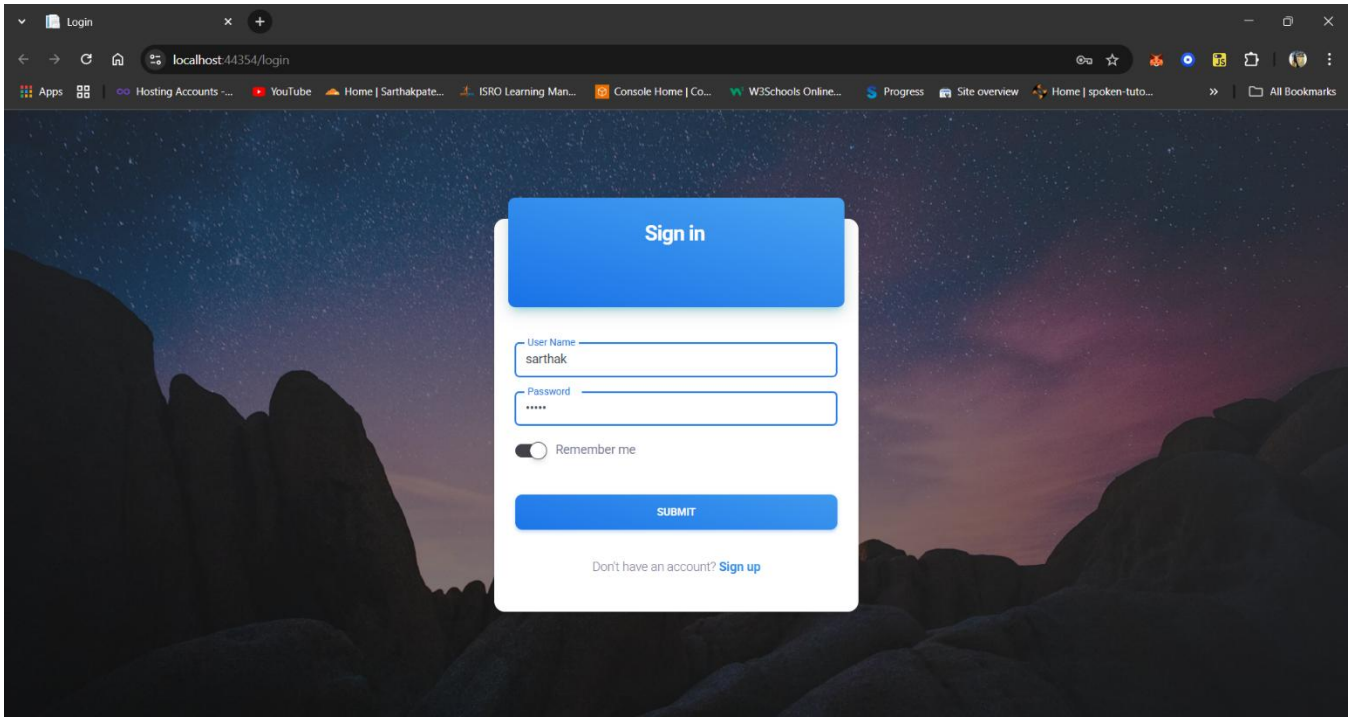

Figure 1: Login Page — Users enter their username and password to authenticate

5.2 Admin — Menu Category Management

The Manage Menu Category page allows admins to define the top-level categories of the restaurant menu (e.g., Drinks, Italian, Panjabi, South Indian). Each category can be assigned a display rank to control its order on the ordering screen, and toggled active/inactive without deletion.

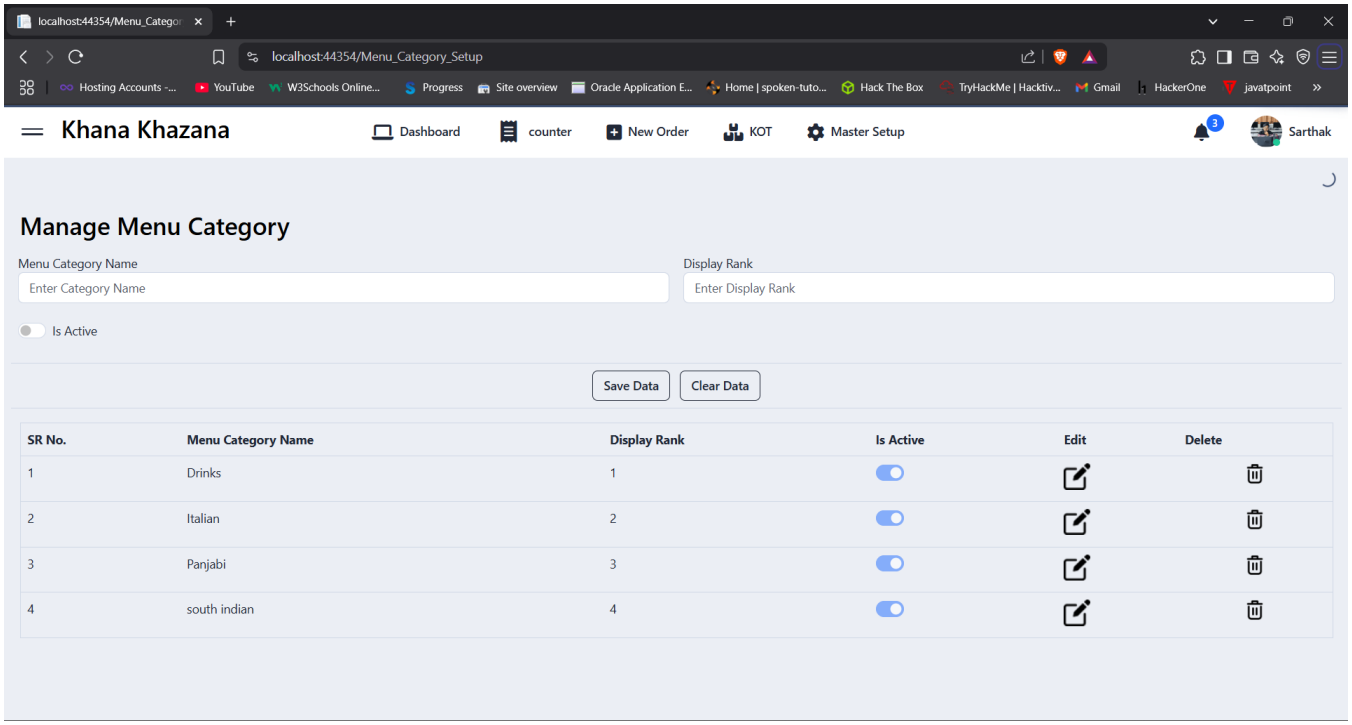

Figure 2: Manage Menu Category — Add, edit, and toggle categories with display rank control

5.3 Admin — Menu Item Management

The Manage Menu Items page provides a comprehensive form to add new items with their name, category, price, display rank, VEG/Non-VEG indicator, add-on and variant flags, and a food image upload. A live image preview appears in the top-right corner before saving. The system validates the uploaded file to ensure only image formats (JPEG, PNG, GIF) are accepted.

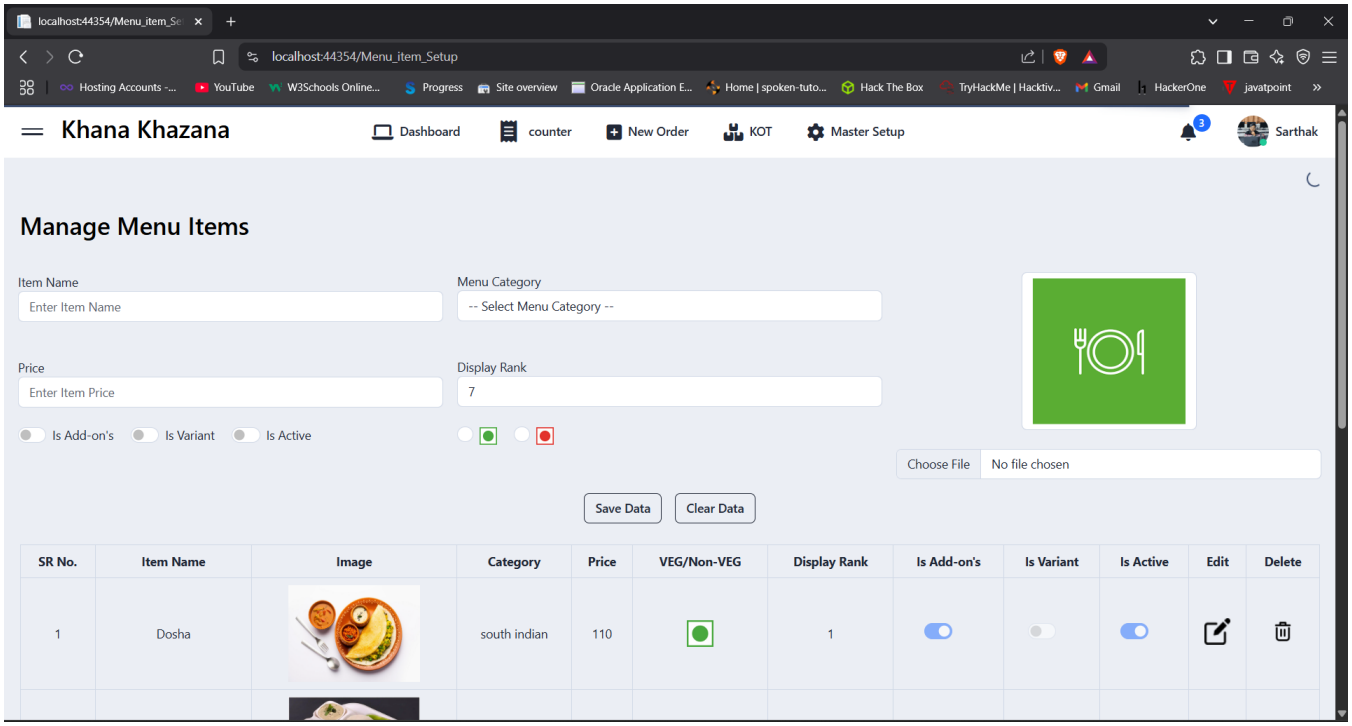

Figure 3: Menu Item Form — Clean data entry with live image preview and category dropdown

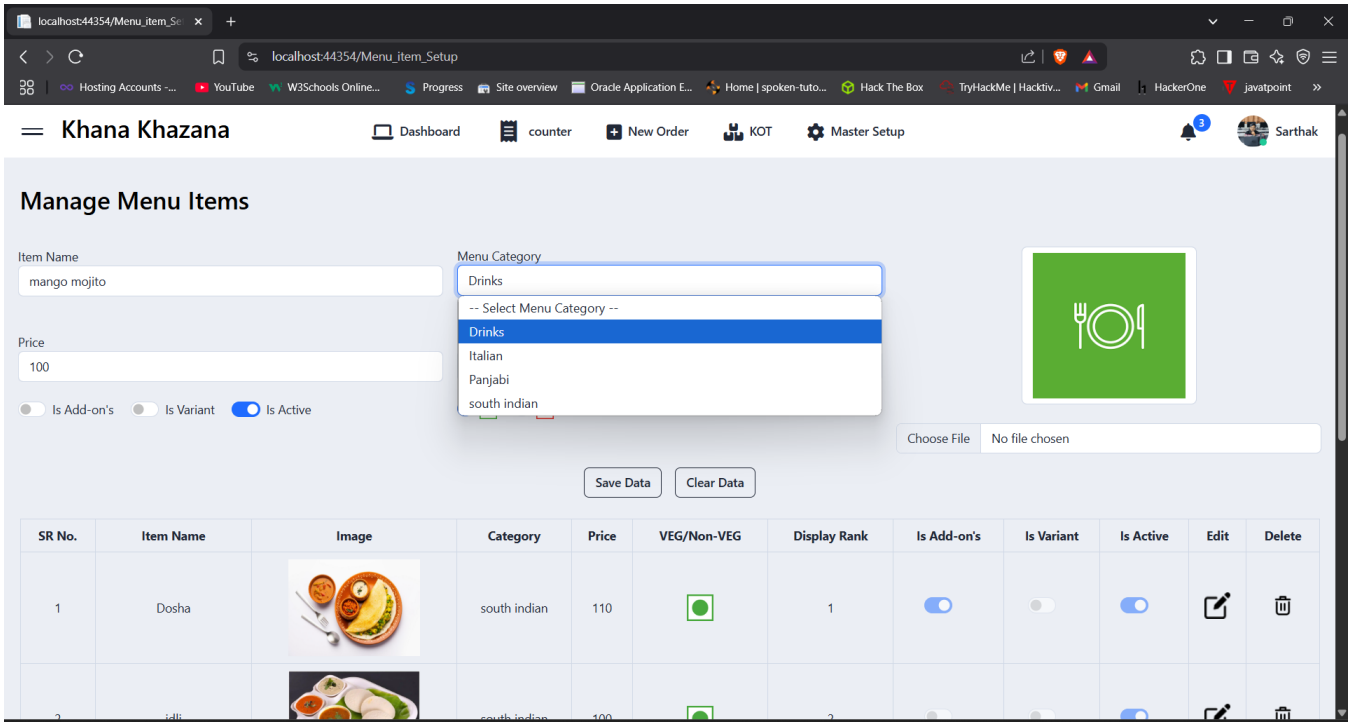

Figure 4: Category Dropdown — Searchable category selector showing all active categories

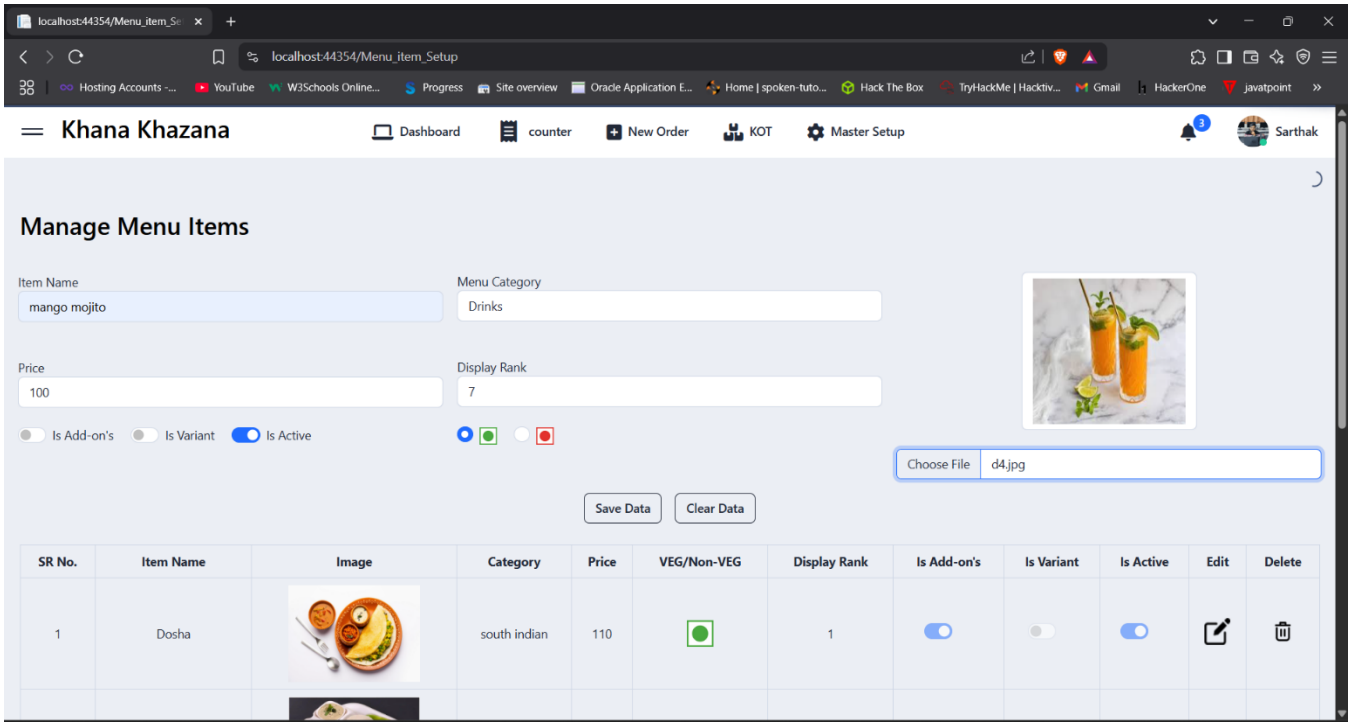

Figure 5: Image Upload Preview — Selected image previewed before saving to the database

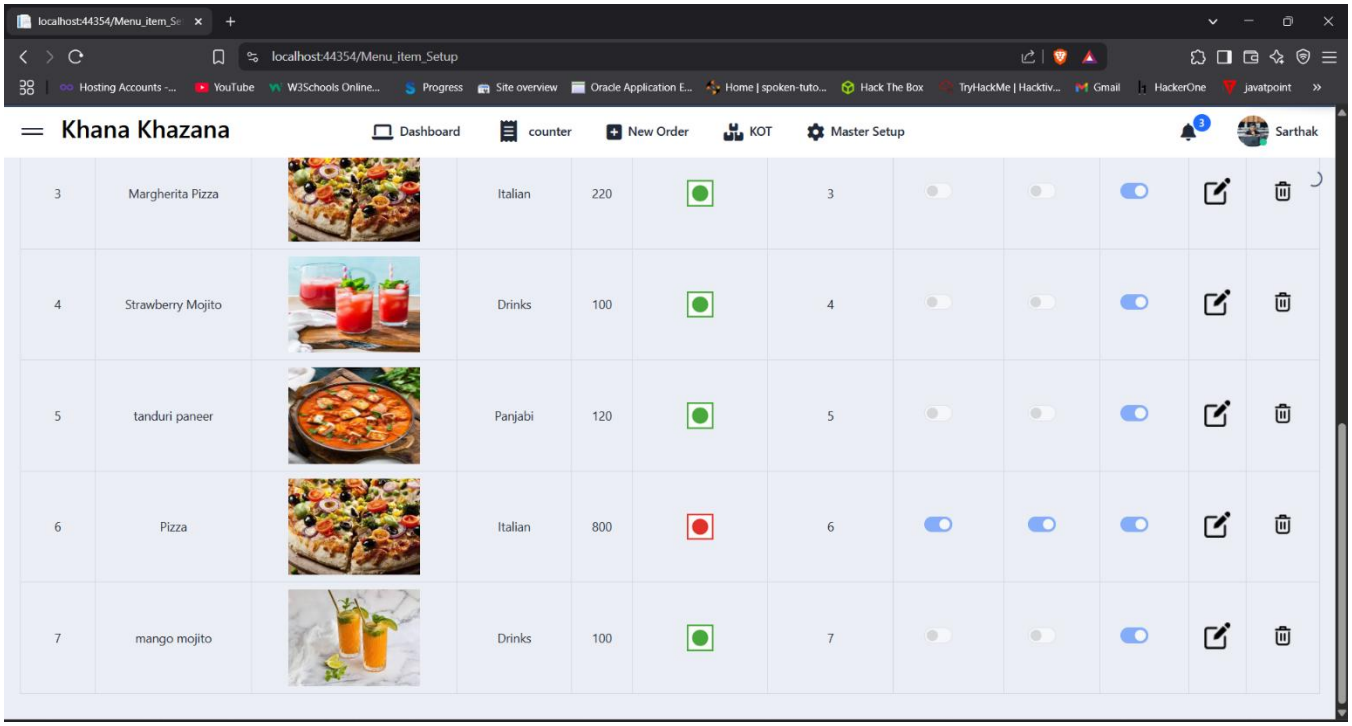

Figure 6: Populated Menu Items Table — All items listed with image, category, price, VEG indicator, and action controls

5.4 Admin — Master Setup Navigation

The Master Setup dropdown menu provides quick navigation to all administrative configuration sections: Tables, Table Category, Menu Item, Menu Category, and Manage User. This centralises all setup operations under a single navigation hub.

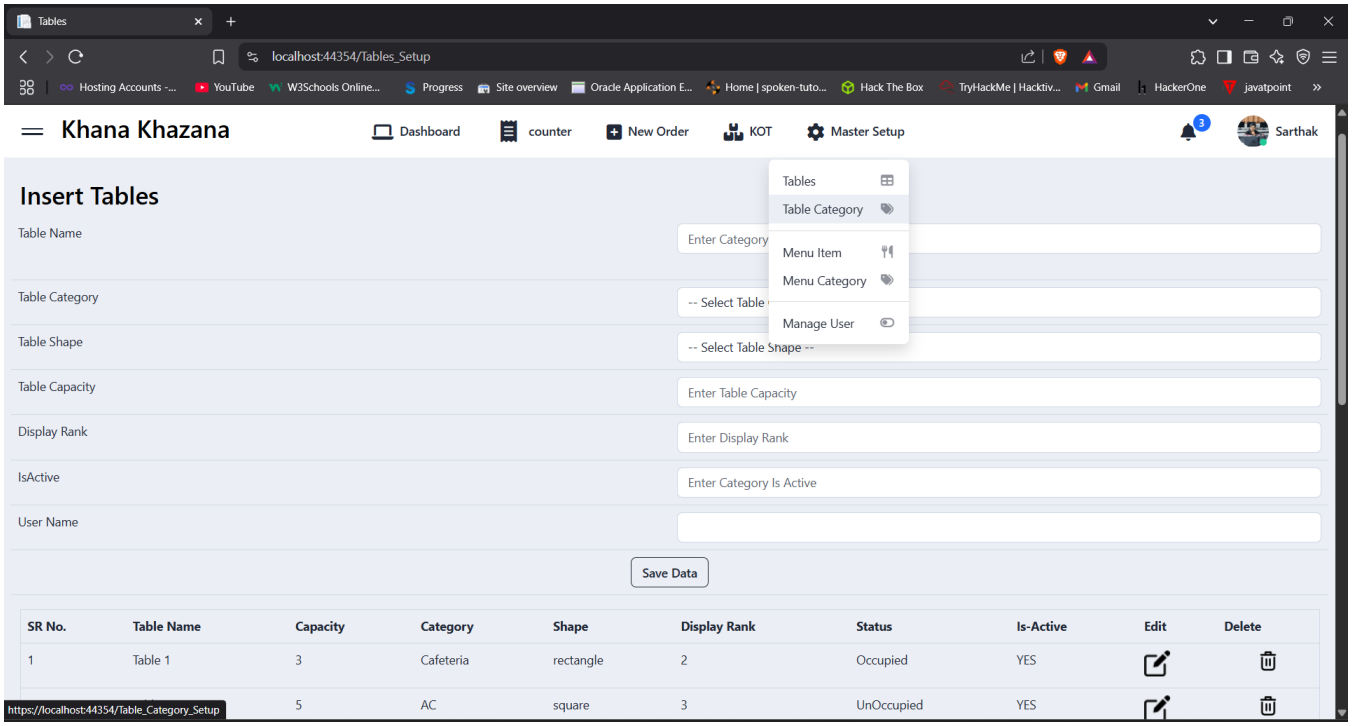

Figure 7: Master Setup Dropdown — Centralised navigation hub for all admin configuration modules

5.5 Admin — Table Category Management

The Insert Table Category page allows admins to create seating zone categories such as Cafeteria, Garden, Family, AC, and VIP. Each category has a display rank and active status. These categories group the tables displayed to waiters and cashiers on the table map.

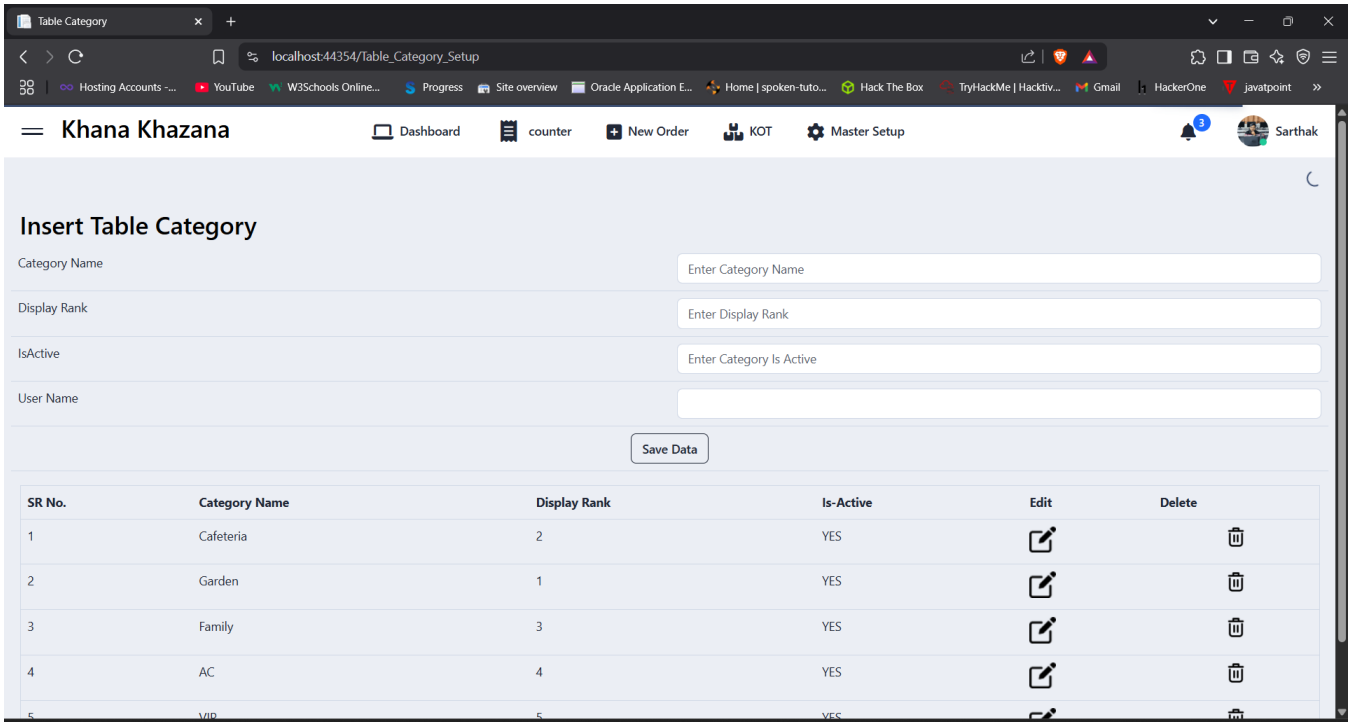

Figure 8: Table Category Setup — Define and manage seating zones with rank and active status

5.6 Waiter — Table Map View

On login, the waiter sees a categorised table map showing all tables grouped by zone (Garden, Cafeteria, Family, AC, VIP). Each table card displays the table name, number of customers vs capacity, elapsed time, total order value, and payment status. A colour-coded dot indicates table occupancy status.

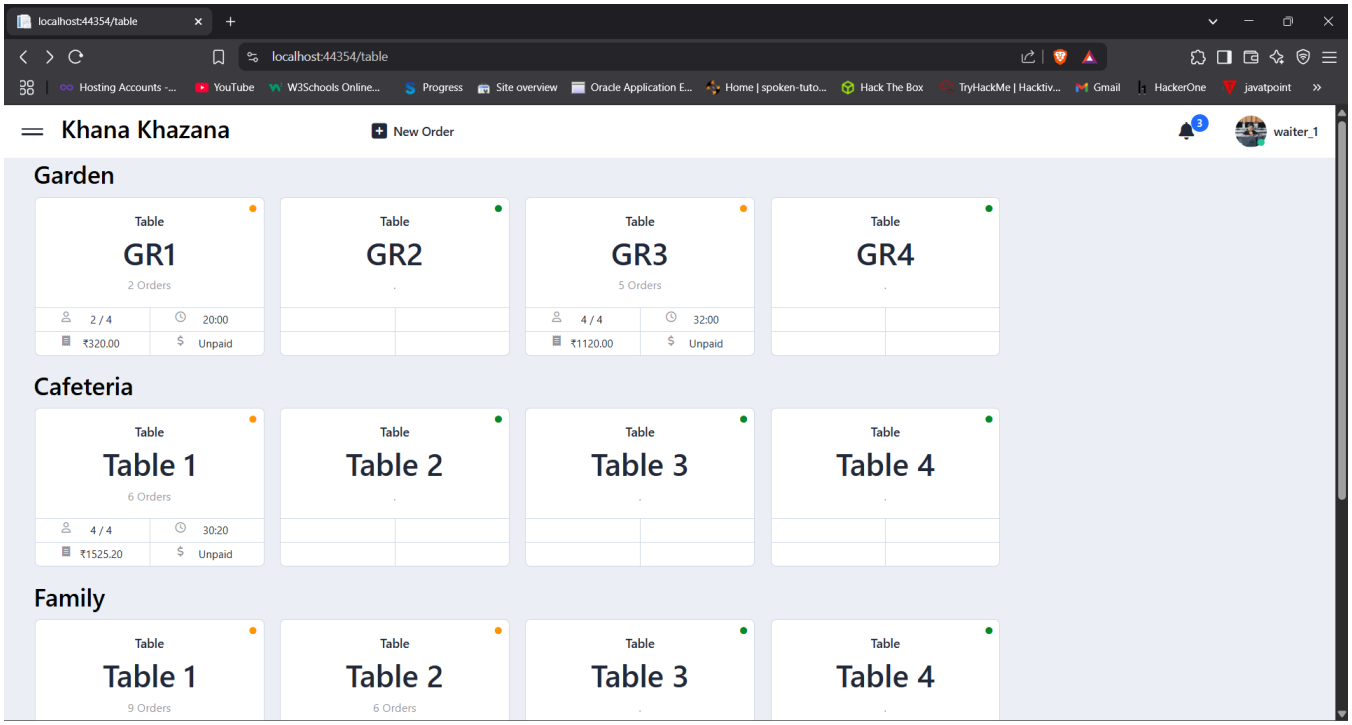

Figure 9: Waiter Table Map — Garden and Cafeteria zones showing live order data and occupancy status

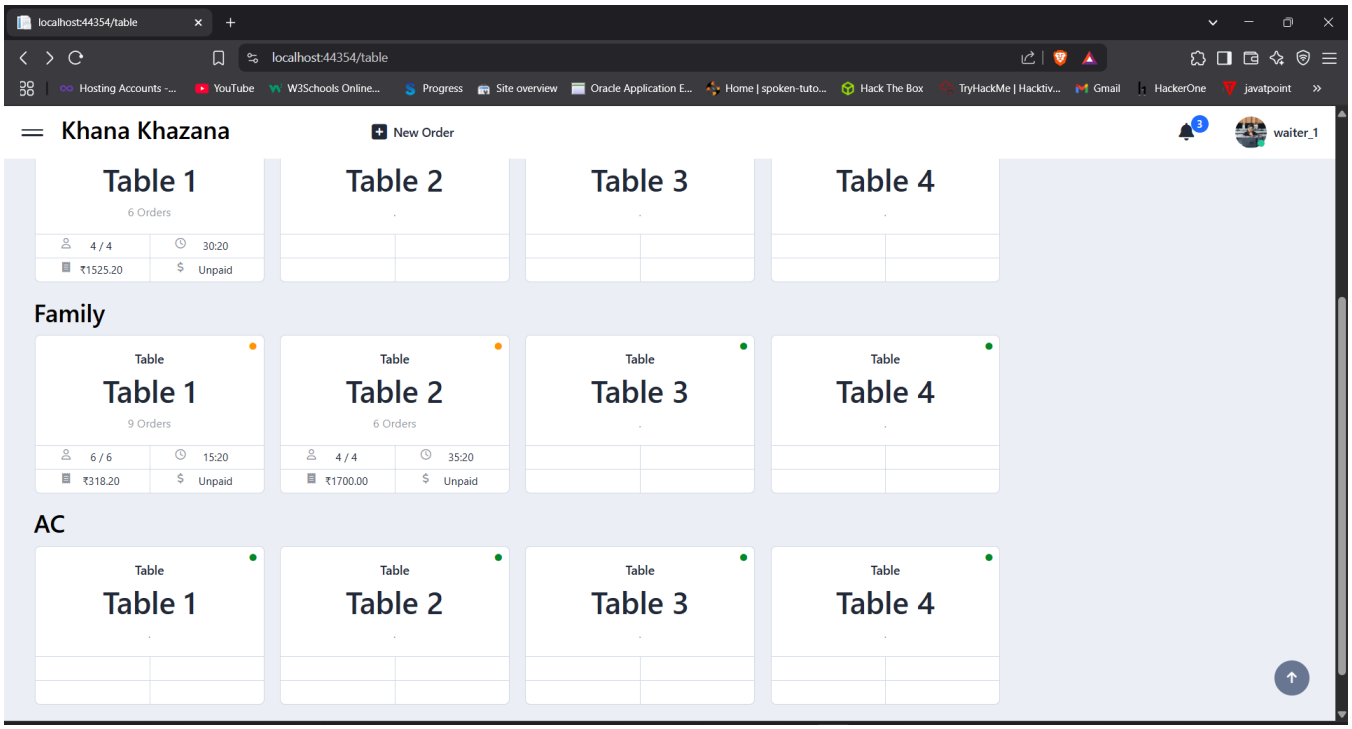

Figure 10: Waiter Table Map (continued) — Family and AC zones; green dots indicate available tables

5.7 Waiter — Order Placement Interface

When a waiter selects a table, the ordering screen opens. The left sidebar shows menu categories as icon buttons. The main area displays food items as image cards with their name, category, and price. The right panel provides a live order summary showing selected items, quantities, subtotal, 5% tax, and the running total. Three action buttons — Service, Bill, and Submit Order — control the order lifecycle.

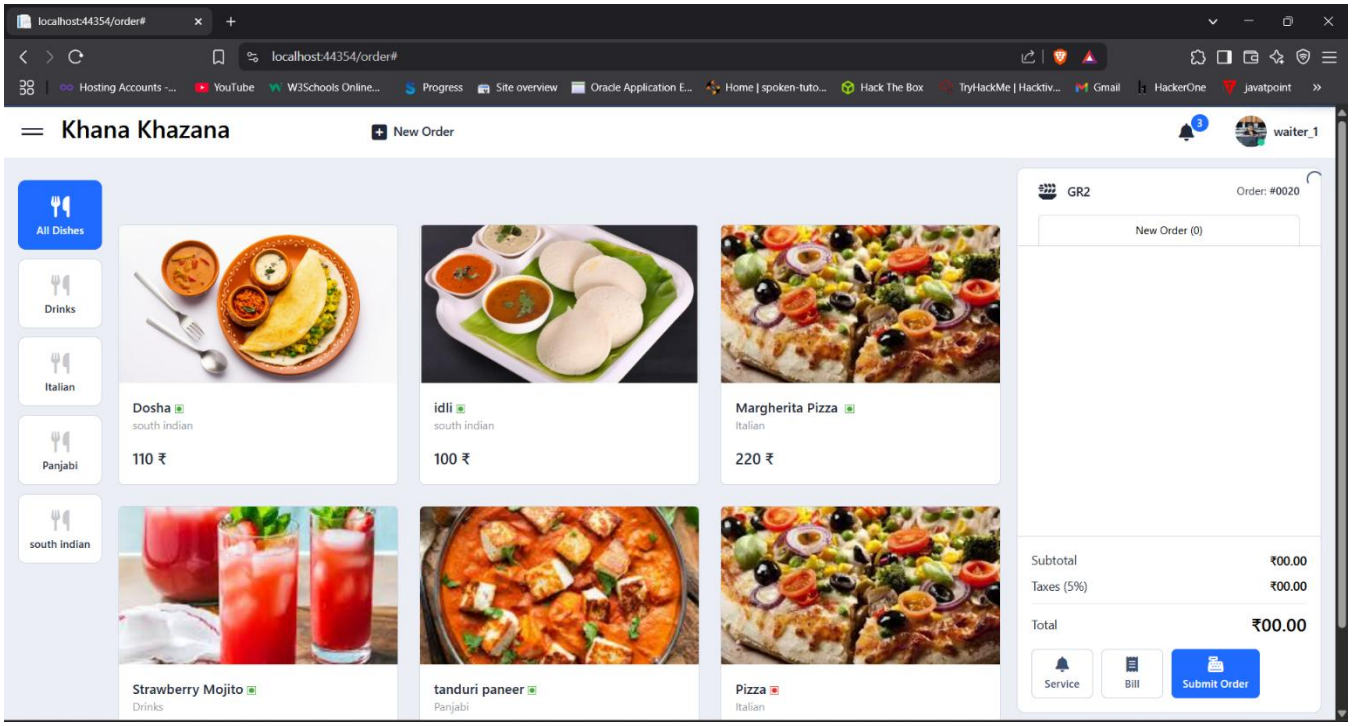

Figure 11: Order Interface (All Dishes) — Full menu grid view with live order cart on the right

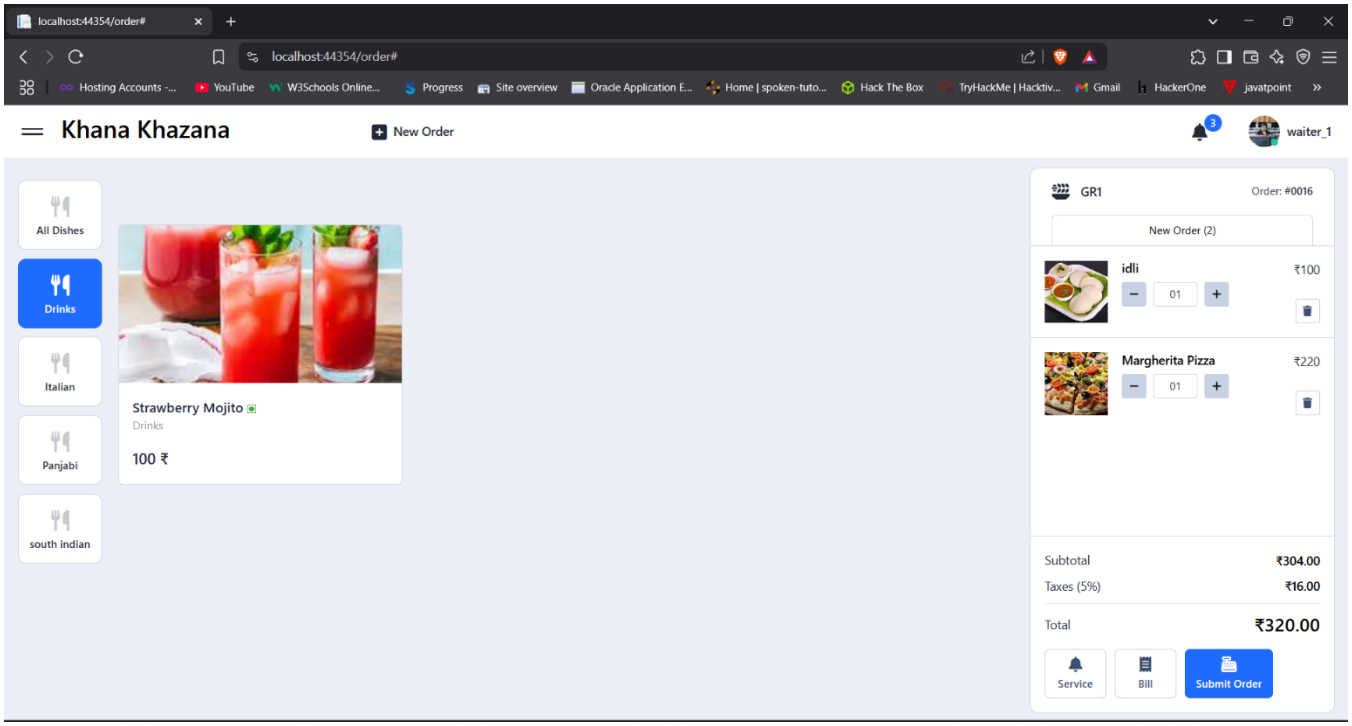

Figure 12: Order Interface (Drinks Filter) — Category filter showing only Drinks items (Strawberry Mojito)

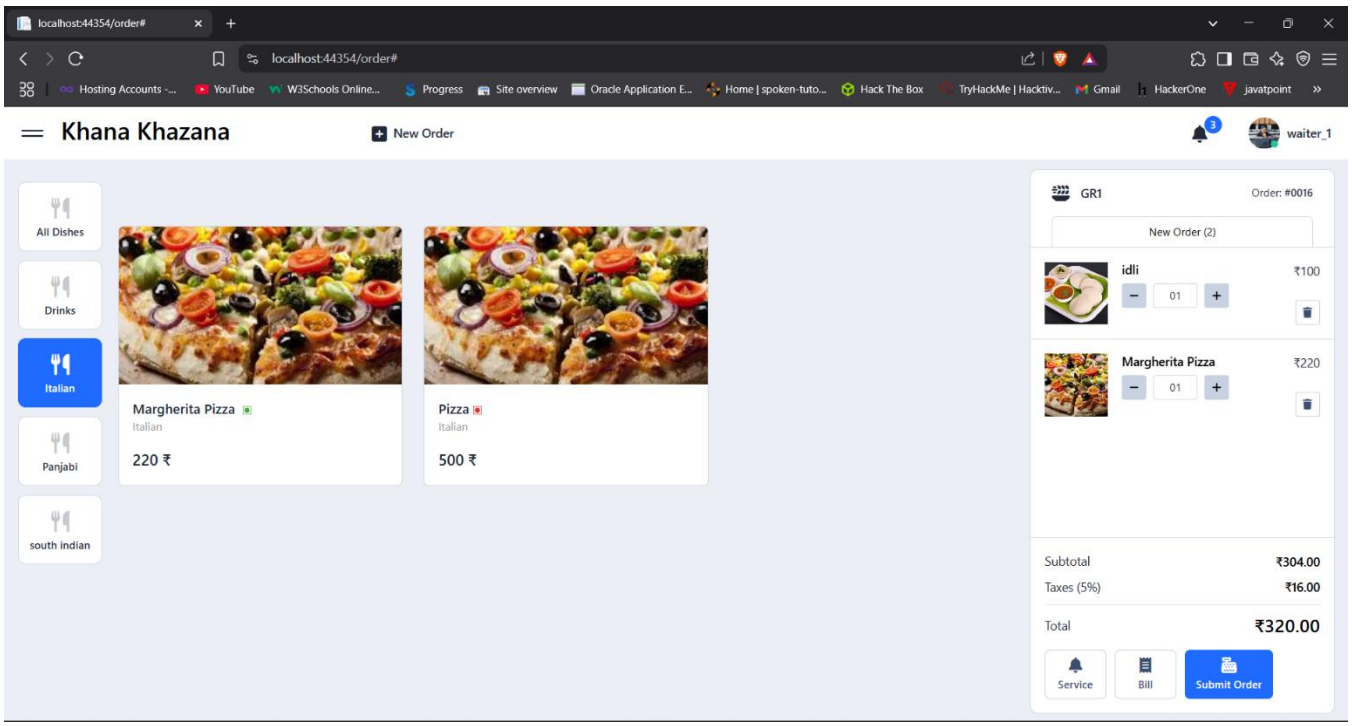

Figure 13: Order Interface (Italian Filter) — Category filter showing Italian items (Margherita Pizza, Pizza)

5.8 Counter / Cashier Interface

The Counter view (accessed by Cashier role) displays the same table map but with an additional right-side billing panel. When a table with an active order is selected, all ordered items appear in the panel with quantities and prices. The panel calculates Subtotal, Taxes (5%), and Total in real time. Three payment buttons — E-Wallet, Bill, and Cash — allow the cashier to finalise the transaction.

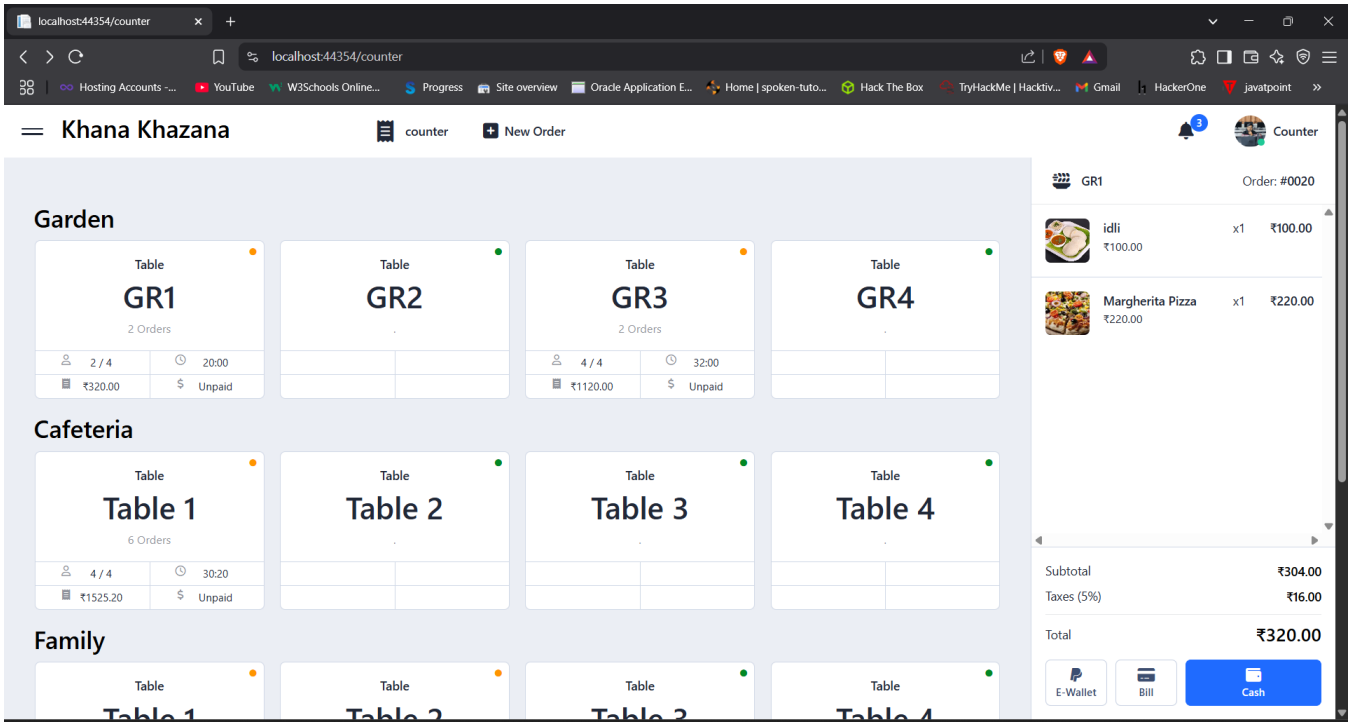

Figure 14: Counter Interface — Table map with live billing panel showing E-Wallet, Bill, and Cash payment options

6. Database Design

6.1 Key Entities

Table Name	Purpose	Key Columns
Users	Stores all system users	UserID, UserName, Password (hashed), Role
MenuCategory	Menu item groupings	CategoryID, CategoryName, DisplayRank, IsActive
MenuItem	Individual food/drink items	ItemID, ItemName, CategoryID, Price, ImagePath, IsVeg, IsAddon, IsVariant, DisplayRank, IsActive
TableCategory	Seating zone groupings	TableCategoryID, CategoryName, DisplayRank, IsActive
Tables	Individual restaurant tables	TableID, TableName, TableCategoryID, Capacity, Shape, DisplayRank, IsActive
Orders	Order headers per table	OrderID, TableID, UserID, OrderDate, TotalAmount, PaymentStatus, PaymentMode
OrderItems	Line items per order	OrderItemID, OrderID, ItemID, Quantity, UnitPrice, LineTotal
KOT	Kitchen Order Tickets	KOTID, OrderID, ItemID, Quantity, KOTStatus, CreatedAt

6.2 Key Relationships

- MenuItem → MenuCategory (many-to-one): Each item belongs to one category
- Tables → TableCategory (many-to-one): Each table belongs to one zone
- Orders → Tables (many-to-one): Multiple orders can exist per table session
- OrderItems → Orders (many-to-one): Each order contains multiple line items
- OrderItems → MenuItem (many-to-one): Each line item references a menu item
- KOT → Orders (many-to-one): KOTs are generated per order for kitchen view

7. Testing

7.1 Test Summary

The system was tested across unit, integration, boundary, exception, and performance dimensions. The table below summarises results across all test cases:

Test ID	Test Case	Input	Expected Outcome	Actual Outcome	Result	Layer
TC-01	Unit Isolation	Individual method	Function works in isolation	Unit works correctly	PASS	BLL / DAL
TC-02	Error Handling	Invalid inputs	Errors handled gracefully	Errors handled properly	PASS	BLL
TC-03	Data Validation	Incorrect data	Validation errors returned	Validation errors shown	PASS	BLL / UI
TC-04	Code Coverage	Unit tests for all methods	Full function coverage	Partial coverage only	FAIL	BLL / DAL
TC-05	Boundary Testing	Edge case inputs (min/max)	Correct boundary handling	Correct boundary handling	PASS	BLL
TC-06	Exception Testing	Force exceptions	Exceptions caught and managed	Exceptions caught properly	PASS	BLL / DAL
TC-07	Performance	Stress test on unit	Response within acceptable time	Performance delay observed	FAIL	Presentation
TC-08	Reusability	Reuse function elsewhere	Function behaves identically	Reuse successful	PASS	BLL

7.2 Test Observations

- 6 out of 8 test cases passed, giving an overall pass rate of 75%.
- TC-04 (Code Coverage) failed due to time constraints — full test coverage across all BLL and DAL methods was not achieved within the project timeline.
- TC-07 (Performance) flagged a delay in the Presentation Layer under simulated stress conditions. The bottleneck is linked to un-cached menu and table data being fetched from the database on every page load. This is earmarked for resolution via a caching layer in the next version.
- All security-critical tests (SQL injection via parameterised queries, role-based access) passed during manual security review.

8. Future Enhancements

8.1 Planned Features

KOT Completion Workflow

Extend the kitchen interface to allow chefs to update the status of each KOT item individually (Received → In Progress → Ready). This will give front-of-house staff precise visibility into preparation progress and enable smarter table-side service timing.

Inventory Management Module

Build a full inventory module that tracks raw materials and finished goods. Inventory quantities will auto-decrement with each completed order. Low-stock alerts via notifications will help managers reorder before stock-outs occur.

Customer Self-Ordering Website

A public-facing web portal or QR-code-linked mobile page where customers can browse the menu, customise their order, and place it directly — reducing dependency on waiter availability during peak hours.

Business Intelligence Report Module

- Daily Cash Report: Total inflows/outflows per business day
- Sales Report: Daily, weekly, and monthly aggregated sales with item-level breakdown
- Peak Hours Analysis: Heatmap of order volume by hour and day of week
- Top Selling Items: Ranked list of items by revenue and quantity sold

Caching Layer

Introduce in-memory or distributed caching (e.g., Redis or ASP.NET Output Cache) for frequently accessed, slowly changing data such as the menu and table layout. This will directly address the performance delay identified in TC-07.

Automated Database Backup & Restore

Implement a scheduled backup mechanism within the DAL layer. Backups will run automatically at configurable intervals (e.g., daily at closing). A one-click restore option will protect against accidental data loss or corruption.

Third-Party Delivery Integration

Integrate with Zomato and Swiggy APIs to pull delivery orders directly into the POS system, allowing the restaurant to manage dine-in and delivery orders from a single unified interface.

9. Research & Training Summary

9.1 Training at Desai Software

The developer completed a 6-month internship at Desai Software (11 Nov 2024 – 13 May 2025) during which the following core technical areas were covered:

3-Tier Architecture

Learned to structure enterprise applications into Presentation, Business Logic, and Data Access layers. Understood how this separation reduces coupling, simplifies testing, and enables independent deployment of each layer.

Windows Forms (WinForms)

Built desktop interfaces using WinForms controls (buttons, data grids, textboxes). Implemented event handling and data binding to create responsive desktop UIs that update automatically when underlying data changes.

API Development

Created RESTful APIs in C# to enable communication between the front end and server. Learned request/response lifecycle, JSON serialisation, and how to secure API endpoints with authentication tokens.

Crystal Reports

Generated professional business reports (sales summaries, daily cash reports, inventory snapshots) using Crystal Reports. Learned to deploy reports to a live web server so users can generate and download real-time PDFs from a browser.

9.2 Research Findings — POS Industry

Research Area	Finding	Our Approach
Multi-User Management	Most systems support roles but limit customisation	Four dedicated roles (Admin, Waiter, Chef, Cashier) with purpose-built UIs
Third-Party Integration	Zomato/Swiggy integration is increasingly expected	Planned for future release; architecture ready for API hooks
Inventory Management	Real-time stock tracking reduces waste significantly	Identified as top-priority future enhancement
Waiter App Cost	Many commercial POS systems charge extra for the waiter app	Free, fully responsive waiter interface included at no extra cost
UI/UX Quality	Non-technical staff struggle with complex POS interfaces	Minimal, image-driven UI designed for staff with no tech background

10. Conclusion

Khana Khazana successfully delivers a modern, fully functional Point of Sale system for the restaurant industry. The project demonstrates how a well-structured 3-tier ASP.NET application can serve as the digital backbone of a busy food service operation — replacing paper, reducing errors, and accelerating service.

The system handles the complete order lifecycle: from menu setup and table configuration by the admin, through order taking by waiters, real-time kitchen order management by chefs, to final billing and payment acceptance by cashiers. Each role has a clean, purpose-built interface that minimises training time and maximises operational efficiency.

From a software engineering perspective, the project reinforced the value of layered architecture. The strict separation between the Presentation Layer, Business Logic Layer, and Data Access Layer made the codebase easier to test, extend, and secure. Parameterised SQL queries, input validation at the BLL level, and role-based access control together form a solid security baseline.

The 6-month internship at Desai Software provided real-world exposure to enterprise development practices — from 3-tier architecture and API design to Crystal Reports and live web server deployment. These skills, combined with the Khana Khazana project, form a comprehensive practical foundation for professional software development.

Future enhancements — including inventory management, customer self-ordering, business intelligence reports, and delivery platform integration — are well-defined and architecturally ready. The system as built is a production-grade platform that can be deployed to any restaurant with minimal configuration.

11. Bibliography & References

- Practical Case Study on Restaurant POS System — Internal Desai Software documentation
- C# Corner: https://www.c-sharpcorner.com/
- Microsoft Documentation (ASP.NET, C#, ADO.NET): https://learn.microsoft.com/
- Stack Overflow — community Q&A for implementation challenges: https://stackoverflow.com/
- DrawSQL — database diagram tool: https://drawsql.app/
- Bootstrap Official Documentation: https://getbootstrap.com/
- jQuery Official Documentation: https://jquery.com/